

Prompt engineering in that scenario is just as important as it is for structured output generation. You need clear, small scoped goals for the agent to complete. Prompt engineering extends to how to create your Claude.md document, along with having appropriate skills in the environment.

7.13 Google Antigravity

The last coding tool I will show is Google's Antigravity code editor and agent tool. You can download Antigravity from <https://antigravity.google/download> and install it on your machine. It requires authentication using a Google login (like a gmail account), but otherwise has a free tier for users.

Tip

There is a similar coding agent tool, Cursor <https://cursor.com/>, that is likely more popular among software developers currently than Google's Antigravity (which was just launched in November 2025). Cursor is also a VS Code fork, and allows running agents to execute coding tasks. I do not show Cursor here, as they only have a limited time free trial (which I have already used).

Google was slightly later to the market than many of the competitors I show in this book. (The initial Gemini LLMs were quite low quality relative to OpenAI or Anthropic, but the Google models have caught up in 2025.) So that is one reason they offer aggressive pricing and even free tier usage – to get more developers to switch to their products. Google has deep pockets, but I do not expect that generosity to last forever!

Once you have downloaded the application and signed in, you will be greeted with a screen that looks like below (minus if you chose a different color scheme). If this application looks familiar, it is a fork of VS Code (although I use a darker color scheme for this section to keep the applications straight).

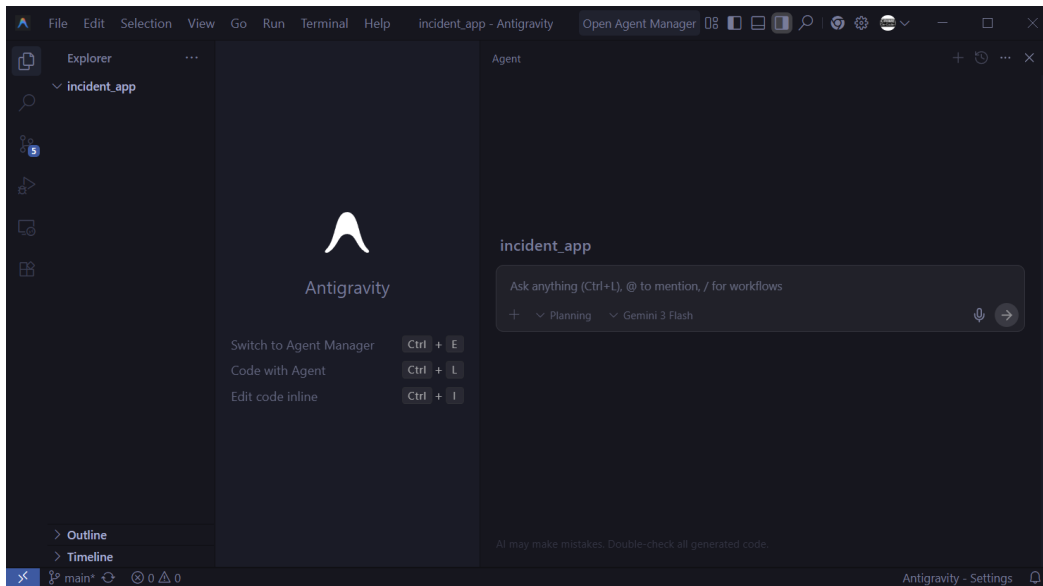


Figure 7.63: Google’s Antigravity Code Editor

In the prior two examples in this chapter, Copilot and Claude Code, I illustrated editing an existing repository. In this example, I am going to show how to build a tool from scratch: a Flask app that helps crime analysts make a custom database to input crime incidents. Coding agent tools are especially adept at creating all the initial files needed for a project application, what developers often call “boilerplate” code.

Tip

While the examples in this chapter show writing computer code, the same LLM agents are capable of other non coding tasks. The same as I have Claude Code write meta-data for functions, I could have Claude Code (or Copilot or Antigravity) write documentation for other materials.

It is easiest to use the coding agents to write documentation in plain text files, like markdown. So learning how to turn markdown into other documents is a simple way for LLM coding tools to help write additional documentation.

First, I opened an empty folder named `incident_app`. I created a detailed prompt on the components of the Flask app, then copy-pasted that prompt into the chat bar on the right-hand side of the window. (I have this prompt saved in the same GitHub repository with the rest of the artifacts for the book). Antigravity, like Copilot, has the ability to switch models, including those from Anthropic and the open source GPT-OSS model from OpenAI.

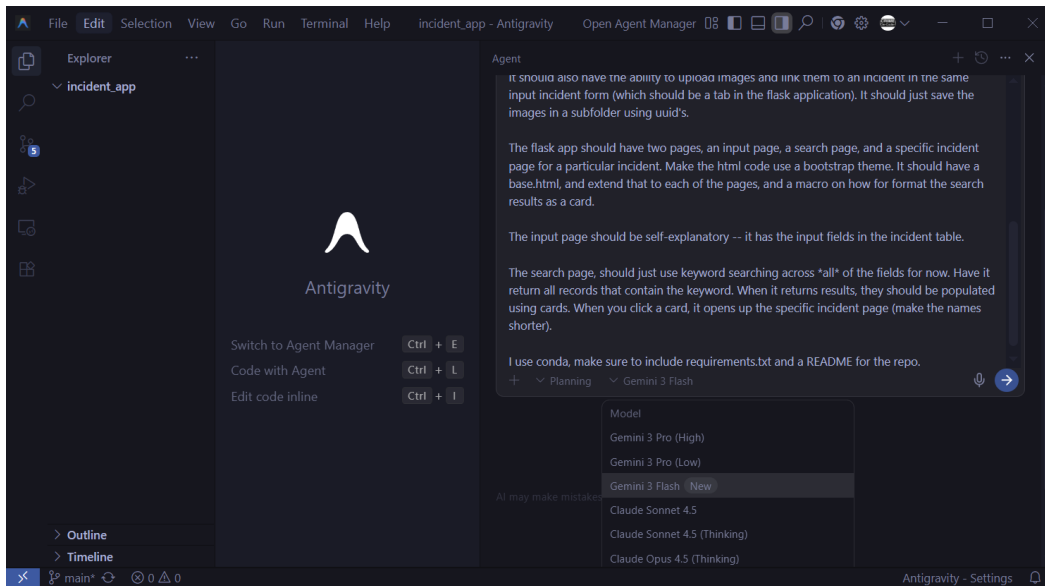


Figure 7.64: Can switch between models

I select Flash (the cheaper Gemini model), and click the submit arrow. By default it creates an implementation plan and task list first before it starts writing any code. You can comment on the task or implementation plan; for example, I added a comment saying the `werkzeug` python library was not needed in the `requirements.txt` file.

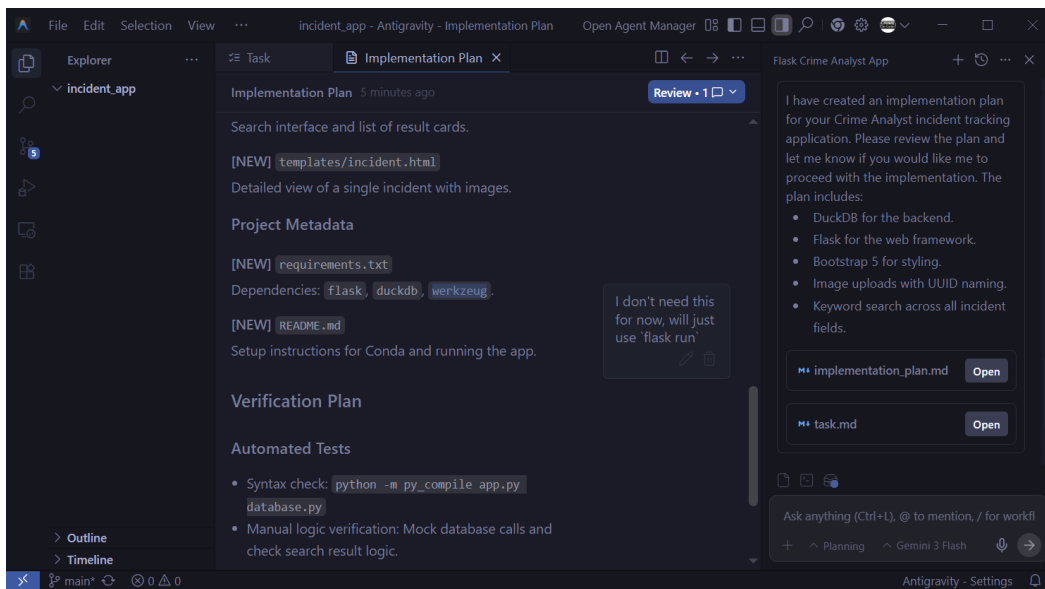


Figure 7.65: Can review the plan and make comments

Tip

The Antigravity editor has many of the same features as those I showed for Copilot earlier in the chapter – such as autocomplete and the ability to use Ctrl+i to open an inline chat application.

Then once you have reviewed these files to your liking, in the chat application on the right you can just say “run” for Antigravity to start editing the code. This will then kick off writing the code. When the LLM tool needs permission, like here it is asking to create a new folder, it will pop up a notification, and you can hit “Accept” to let the tool continue working.

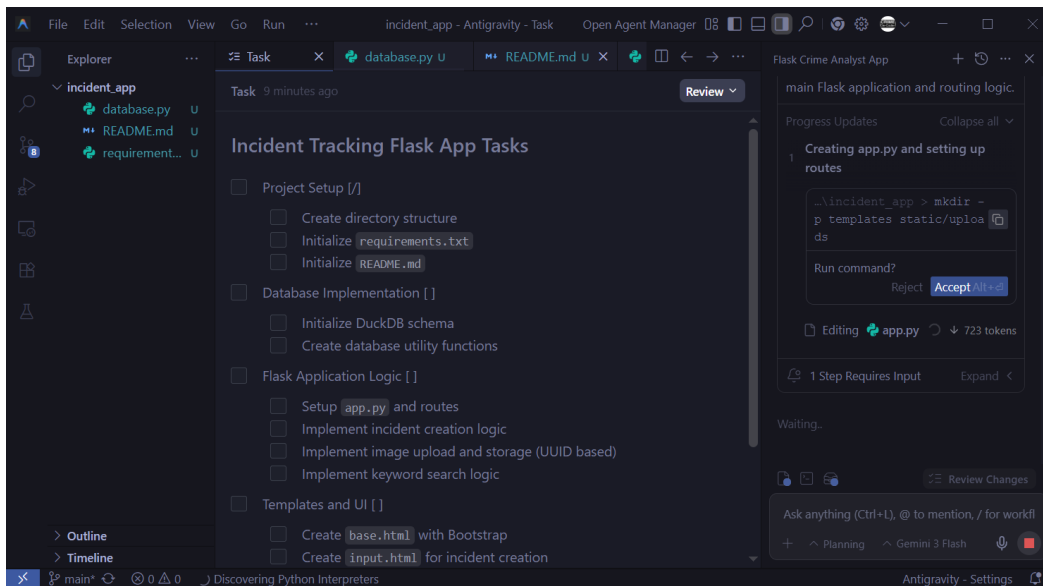


Figure 7.66: By default need to approve making a new directory

At the top of the window, there is text that reads “Open Agent Manager”. You can also press Ctrl+e to open this same window. Go ahead and open that agent manager window. In the top left, I have an inbox with 1 unread message. (Antigravity has the ability to run many projects at once, so if you have it churning away on multiple projects, this is a central plane where you can see updates for all of the different projects and agents.) When I click that inbox, I can see that the flask app creation agent is waiting for me to approve another step (another mkdir command).

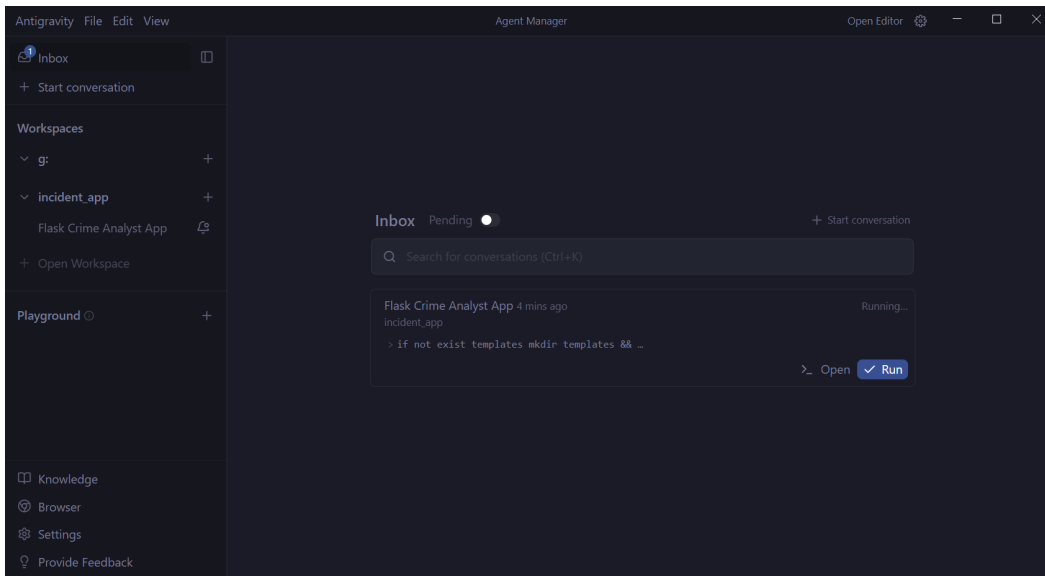


Figure 7.67: Agent manager shows if there are tasks that need your approval

You can either click to review it, or just accept right in the agent manager window. I just clicked Run in the agent manager window, and then a few seconds later was asked a new question to run a set of python code:

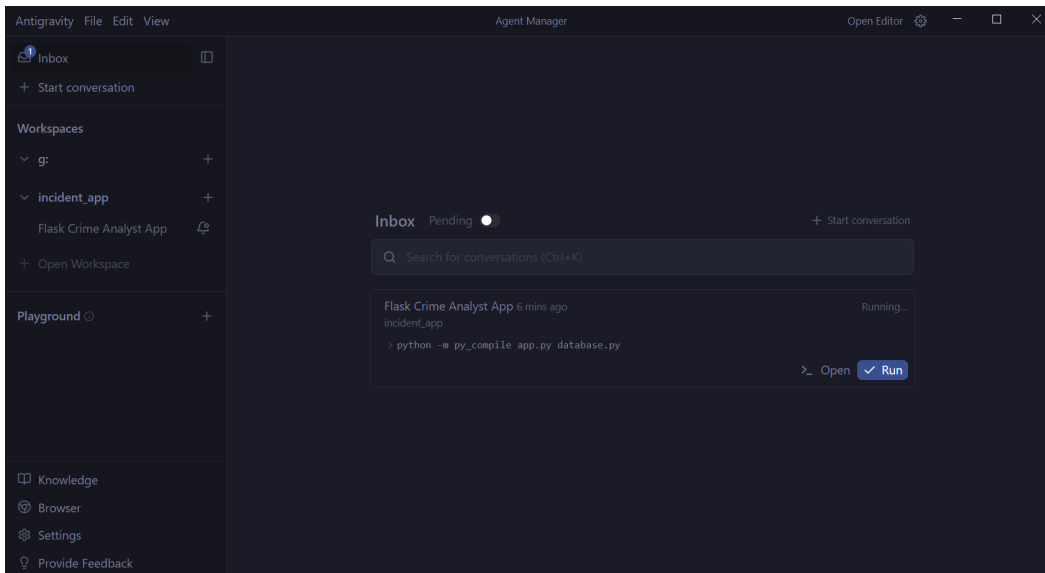


Figure 7.68: Follow up to compile the python app

I am going to review this code first before running. So I go back to the original window, and open `app.py` to review. You can directly edit these files without using the LLM to edit the files. For a simple example, I just manually take out the `gif` image option from the allowed extensions list.

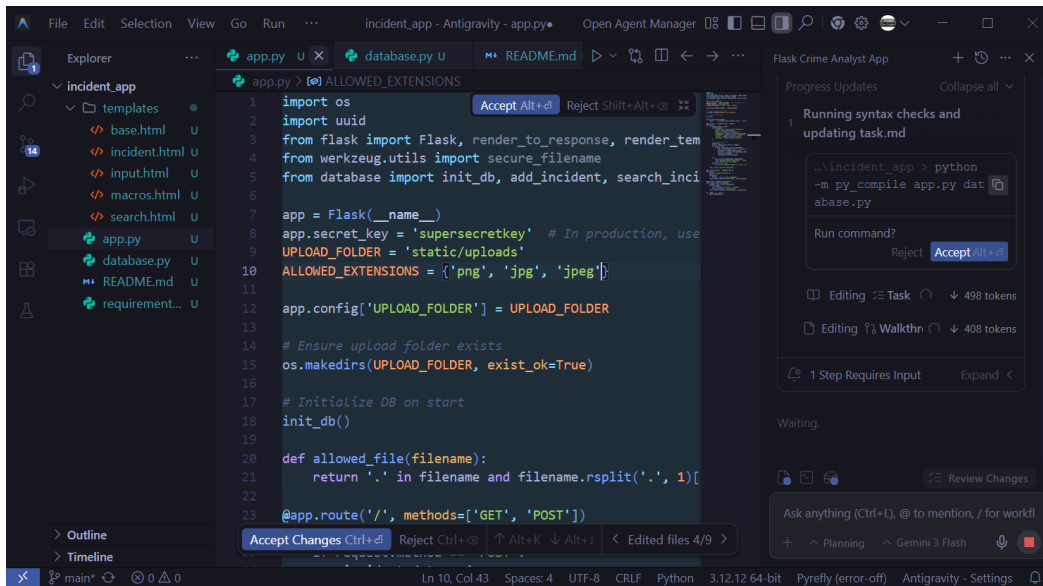


Figure 7.69: Can manually edit files

I hit Ctrl+s to save the edits, and then hit “Accept” to execute the `python` command. Afterward, a “Complete” popup appeared, that says the agent task is complete, and then noting that nine files had been created or modified.

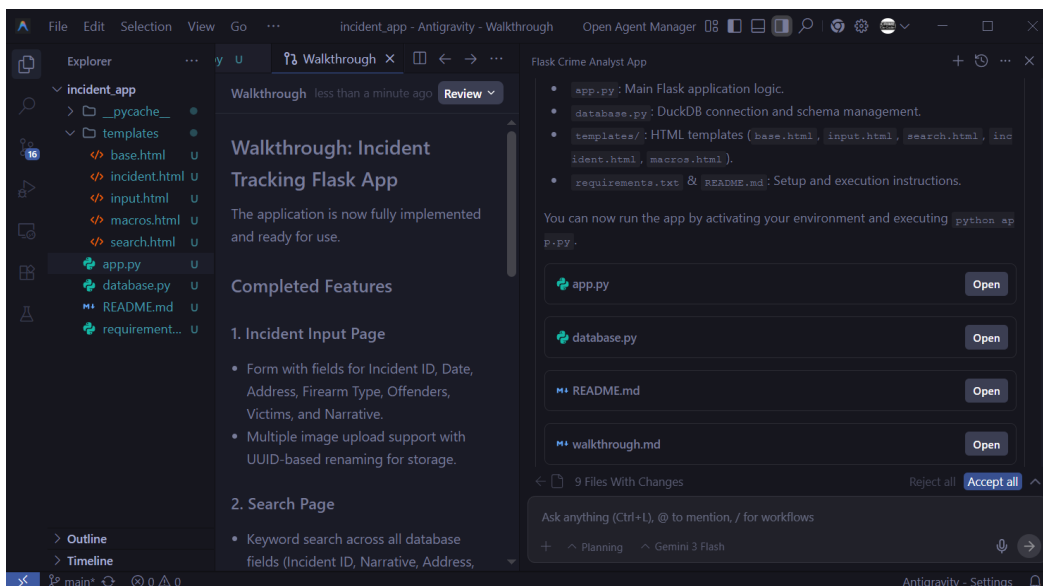


Figure 7.70: Can accept all changes at once

The flask app, especially at this early stage, is difficult to test one file at a time. So it is easier to just accept all and then edit if the files have any particular errors. I’d note these tools are not perfect, if I open up my requirements.txt, it includes the `werkzeug` I said not to include.

7 Coding Tools and AI-Assisted Development

Next, I opened a terminal (just as in VS Code). I created a virtual environment using Conda, activated it, and attempted to spin up the Flask application. I encountered an error though; a function in the generated code does not exist in the Flask library.

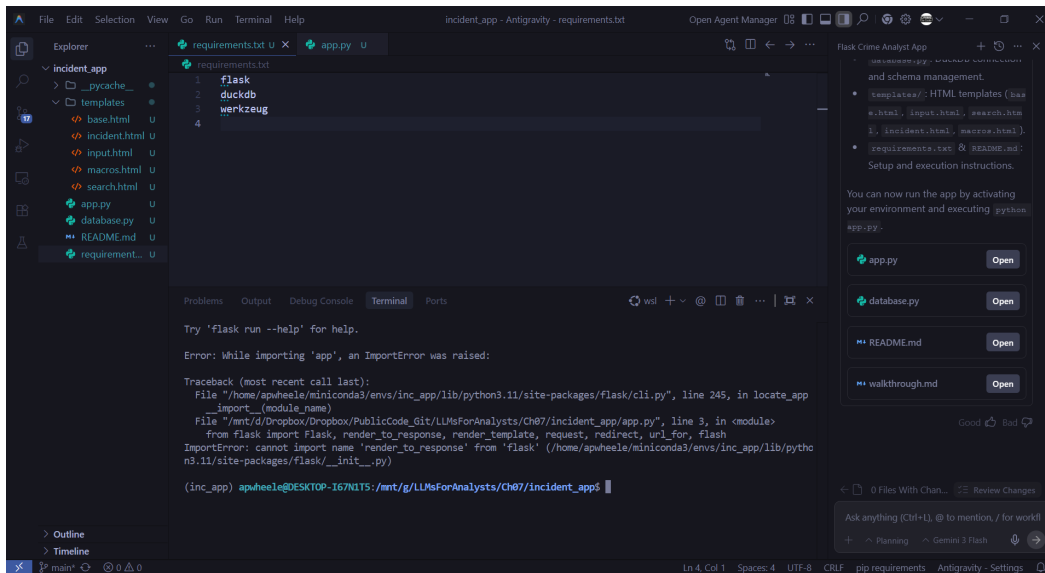


Figure 7.71: The Flask code has an error

This happens to be a version difference – the function is for an older version of flask, and the newer version does not have this function. (It is a common error, especially when asking the foundation models to write code to use their own foundation model APIs, that they write out of date code.) I went back over to the agent, and told it to fix the error, but first, I changed the conversation text box from “Planning” to “Fast”.

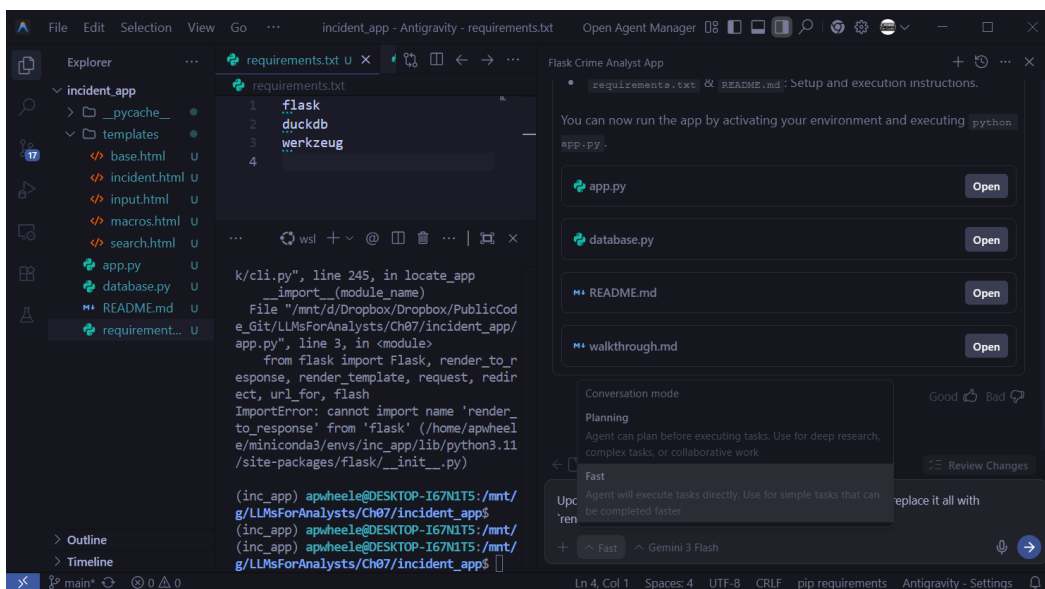


Figure 7.72: Changing from planning to fast mode

This changed both the requirements.txt file to pin to a newer version of Flask, as well as edited app.py. When I run the code `flask run` now, it works, and I can open it up on localhost to see what it looks like. In the top right of VS Code, there is a small Chrome Icon (a few icons to the left of the minimize button). Once you hit that, you will get a screen asking to install an extension to use the browser tool.

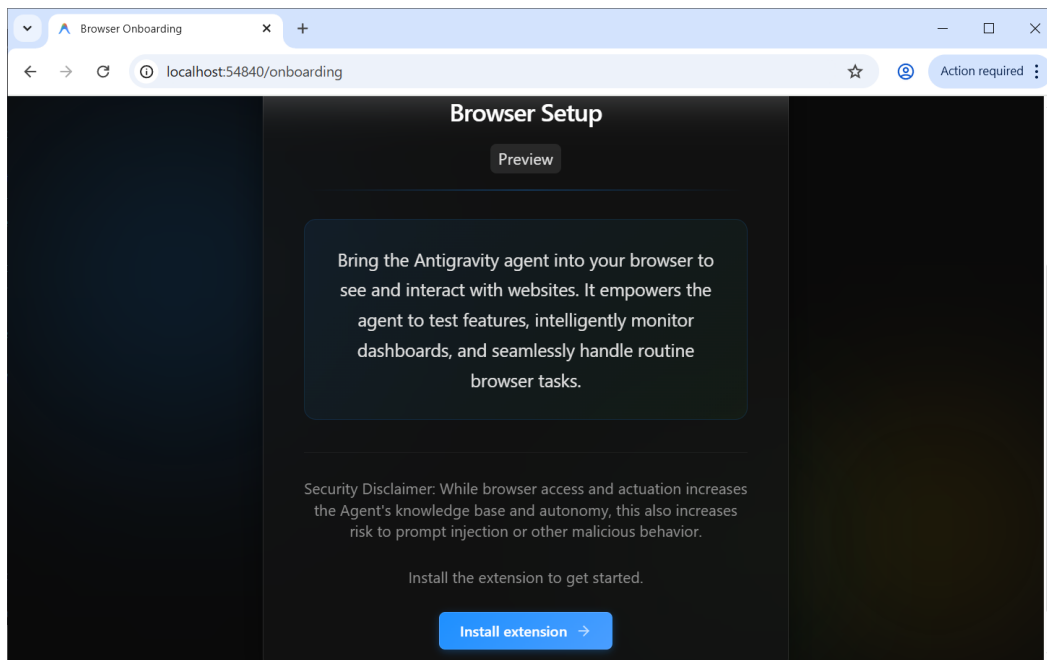


Figure 7.73: Setting up browser extension

While it gives a warning about prompt injection, the risk is really no different than when using the web search tools with Copilot or Claude Code (do not sign into your bank account in this browser)! I clicked “Install extension” and added the extension to Chrome. Now I can go ahead and open my Flask app and see what the initial prototype looks like.

Figure 7.74: Initial Flask Prototype

Now Antigravity can not only edit the code, it can also interact with the browser.

Tip

Claude Code also has the ability to interact with the Chrome browser via an extension, see <https://code.claude.com/docs/en/chrome>. You can launch Chrome (preferably in an incognito window without important applications signed in), then run the `/chrome` command to interact with the browser. This tool requires an Anthropic subscription, and does not work in WSL.

So far, I have been editing code in my Window's path. Now I switch to the WSL path. Unlike Claude Code, when Antigravity creates a subagent, it does not inherit from the same command line environment, but from the base (here Ubuntu) environment. That is a problem when having the agent interact with the application.

At this point, if you ask the agent to interact with the application, you may encounter errors, such as Python wanting to install packages or create a new environment.

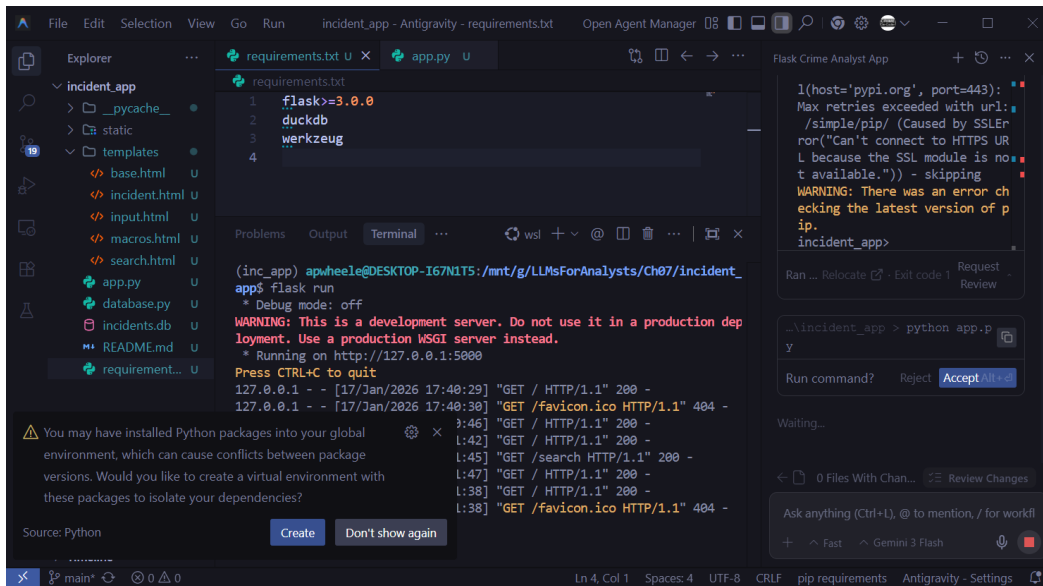


Figure 7.75: The agent is not using the correct environment

To fix this, I told the agent instead of launching the app simply via `flask run` or `python app`, to create a shell script to set up Conda and activate the correct python environment. I then also have the application create a `GEMINI.md` file (the equivalent of a system instruction for Antigravity), where it gives instructions how to spin up the application.

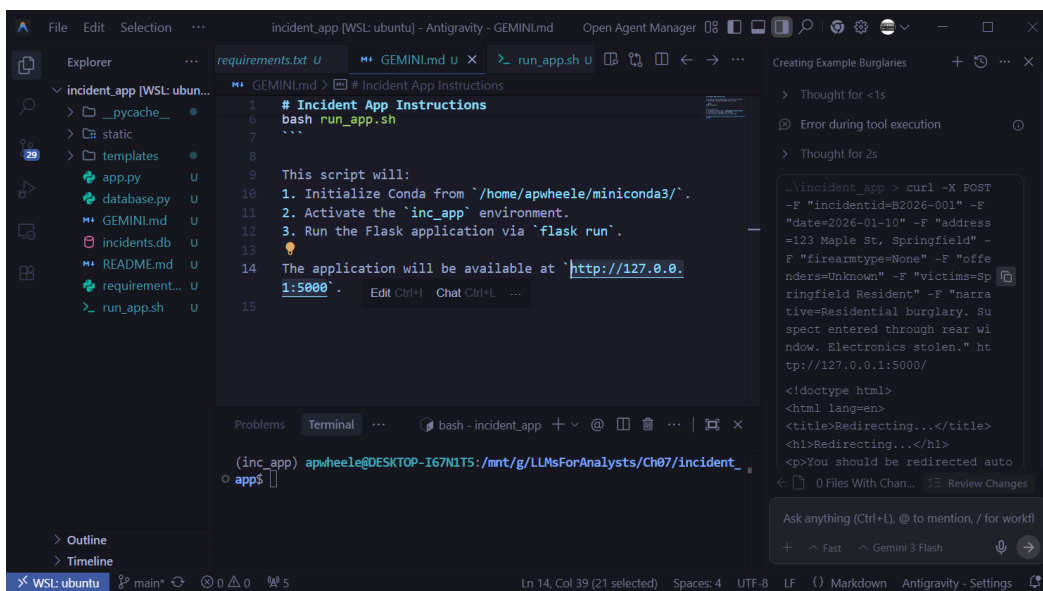


Figure 7.76: Fixing how flask app is started

Now we are ready for the agent to go to work. I ask the agent to input two different burglary incidents into the application. Like boilerplate code, creating fake data to test applications before LLM coding

tools could take a significant amount of time. Now it is as simple as prompting to create some mock data to test.

Antigravity ended up creating POST statements to fill in fake data (as seen in the screenshot above). Now when I manually navigate to the “Search Incidents” page and search for burglary, I can see my two events.

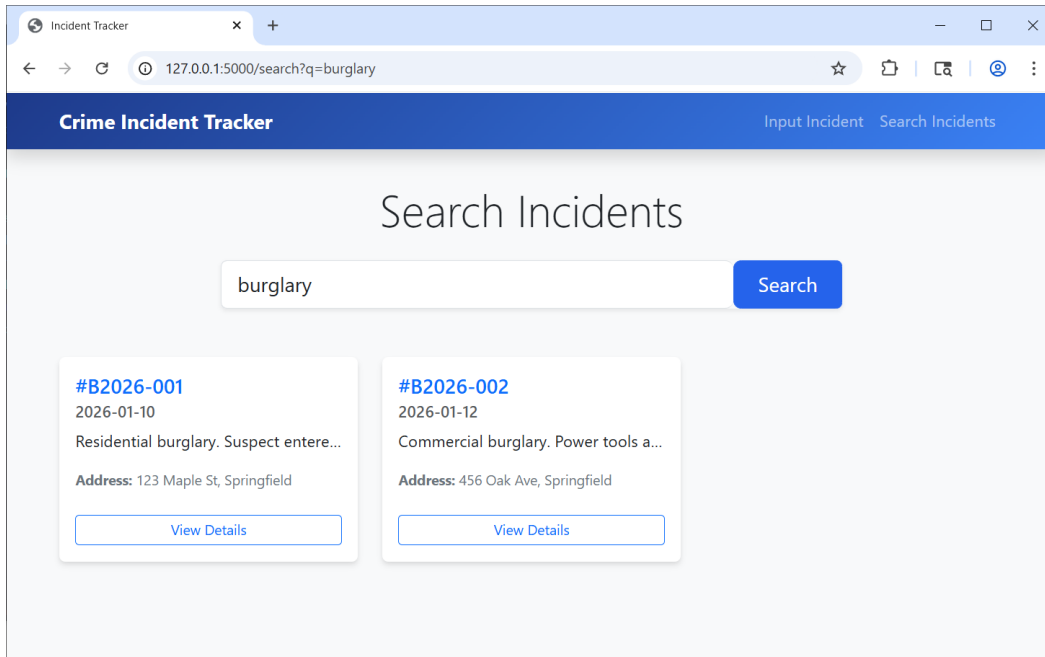


Figure 7.77: Can see my two burglary incidents

If I click on one, I can see what the page looks like for one of the fake incidents. It does not have any associated images with it though.

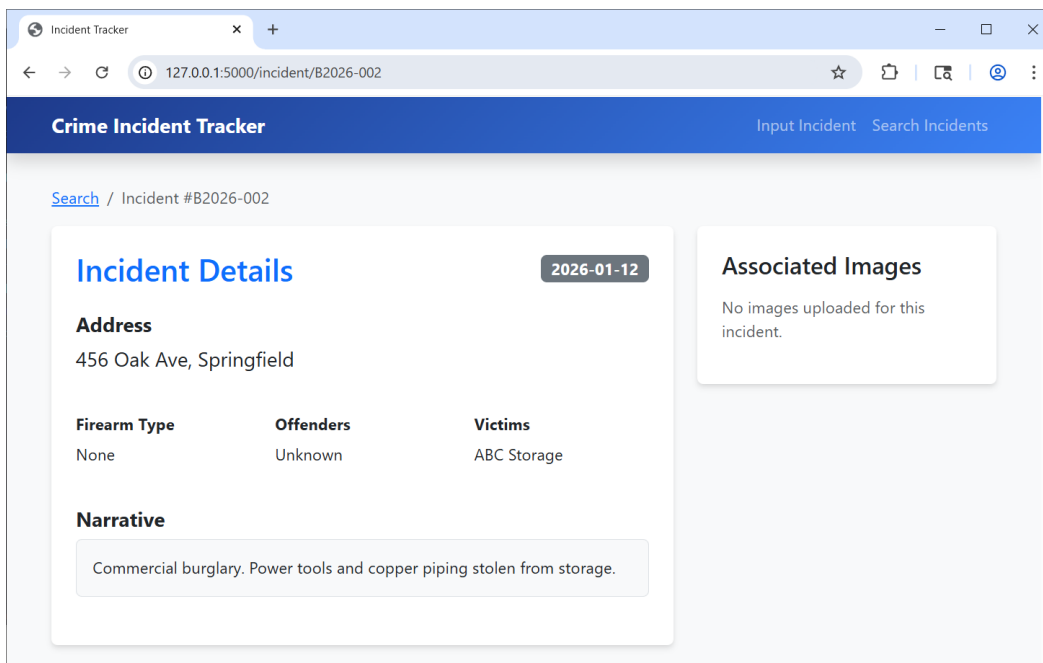


Figure 7.78: Can go into a single incident

To test whether the image functionality was working, I asked the Antigravity chat to “create a theft from a motor vehicle incident, create a fake image of a broken car window in a car to upload in the app.”. The agent uses Gemini’s image creation tools to generate the image and input the test case. Here you can see the final result – it looks good on its first pass.

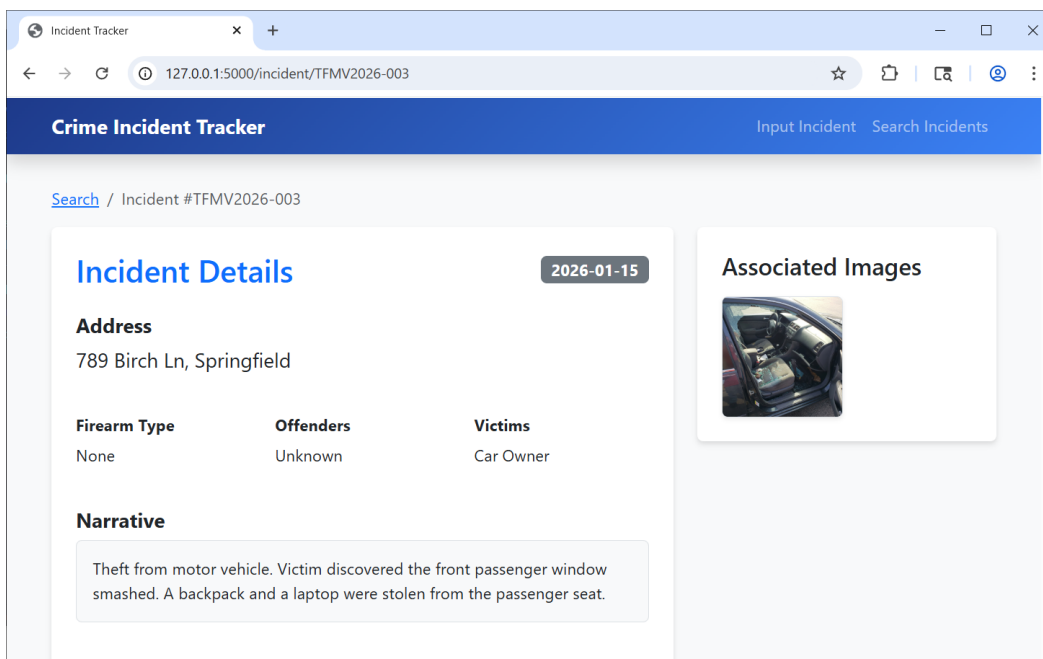


Figure 7.79: Can request images be generated by Gemini

So far, these tests have not really used the browser agent interaction though. Antigravity has been using POST and cURL commands to fill in data. These do not necessarily verify the actual user input works. For example, the form could have faulty javascript that prevents a user from entering a date. The POST endpoints may still work, but a user would be unable to use the form as intended.

I reopened the app in Windows (I cannot get the browser agent to work in WSL), then I opened up a WSL terminal and started the Flask app manually. I then told Antigravity the app is already running, and ask it to input a date.

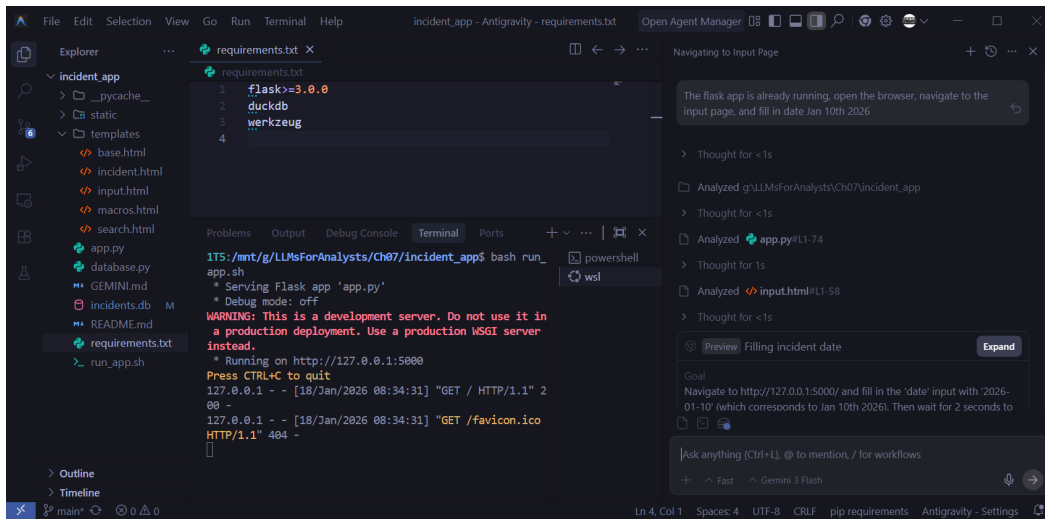


Figure 7.80: Running the flask app manually, and ask agent to interact

It will ask permission for several steps, but you can watch the agent do several steps, like open Chrome, open the app, and then input the date into the form.

Figure 7.81: Browser agent filling in a form field

You can definitely set up Windows environments to run the Flask app as well. So this back and forth between environments is not totally necessary. But hopefully it becomes easier to set up Antigraivty’s browser agent through WSL in the future.

Tip

One way to write tests for the browser interactions is to use the *playwright* library. It can also be useful for automating different tasks that require interacting with web applications when *you cannot* use tools like curl directly.

7.14 Best practices for AI-assisted coding

AI coding assistants are powerful but require good practices. Like I said previously in the chapter, I am not sanguine on the concept of vibe coding with little human oversight. Here I will outline seven pieces of general advice that I think are useful for software developers, especially when they are first learning to use the LLM coding tools.

The first piece of advice is *be specific about requirements*. A more detailed prompt is more likely to result in a desirable outcome than a more vague prompt. “Write this function” may work (Claude Code is quite smart to infer context, so even just the name *may* be sufficient), but “write this function that does XYZ with arguments a,b,c” is likely to be a better prompt. Ditto for tests, e.g. “write a test for function X” is not a good a prompt as “write a test for function X for specific edge cases ... and ...”.

The second piece of advice is *provide context*. Like I showed earlier with Claude Code skills and commands, I provided links to prior examples of code, either in the same repository or in different repositories. Sometimes instead of diving right into writing code, it makes sense to populate the context window with those prior examples. Future code should be guided by those prior examples.

The third piece of advice is *review the generated code*. AI assistants make mistakes. It is easy to just click accept and not really understand what was written. But it is critical to pay attention and really try to understand all the code that was written. It commonly makes logic errors and writes things in ways that are sub-optimal. But if you can read the code and spot the error, it is easy to iterate in a follow up command e.g. “rewrite function X so it is vectorized with numpy”. (Even if I know the code is wrong, I commonly hit accept and then have a follow up command edit to the error I spotted.)

The fourth piece of advice is *iterate incrementally*. It is easier to test and understand code in small pieces. It is also likely easier for the LLM tools to write code in smaller portions. So instead of giving a vague list of many different tasks, just ask the LLM to do one specific task.

If you are at an early stage in the project, and do not know the overall architecture, that to me is a good scenario to use the chat interfaces directly. Ask them to brainstorm about different design patterns, software, etc. Then when you have done your background research, you can move onto using coding agents to build the initial application.

The fifth piece of advice is *test thoroughly*. This fits in directly with reviewing the code and iterating in small parts. If you write unit tests for your code, it makes it much easier to know that making a change does not cause some unintended regression in the code base. While writing tests are not a very fun part of code development, using the LLM tools themselves to help write them makes the process much faster.

The sixth piece of advice is *use version control*. Always commit working code before asking the AI to make significant changes. If the changes break things, you can easily revert to a prior state. Git is really necessary for an analyst to level up to a software engineering role in general. Even without using LLM coding agents, if you are building any sufficiently complicated application you should be using git anyway.

And the seventh piece of advice is *you still need to learn python*. While the LLM assistants can definitely write code from general prompts, you still need to fundamentally understand how the code works. To use some of the extended functionality of the tools, like the browser agent for Antigravity or hooks for Claude Code, I needed to understand how different python environments interacted to set them up correctly. (Part of knowing python is not just writing code, but understanding how to execute it.)

To be able to fill in prompts, you need to know what to actually prompt for. While Antigravity did an excellent job of creating a prototype application, I gave it detailed instructions like database schemas, different pages to create, to use a base template for flask, and to style the app using bootstrap. I only knew about all of those details as I have built flask applications previously.

LLM assistants can be amazing for power users – if you skip the step of really understanding the basics of python though, you will in short order create a mess you cannot untangle, even with simple apps.

The final chapter discusses next steps in your journey – how to stay abreast of new developments and future deep dives into different applications that I only scratched the surface on in this book.